

Javascript – funzioni come dati elementari

In Javascript le funzioni sono considerate come dati elementari. In particolare è possibile:

- Assegnare una funzione ad una variabile. La funzione sarà poi invocabile tramite il nome della variabile
- Passare una funzione come parametro di un'altra funzione. La funzione che riceve il parametro può invocare la funzione che gli viene passata.
- Creare funzioni anonime, ovvero senza nome, utilizzabili senza doverle salvare.

Javascript – funzioni come dati elementari

- Assegnare una funzione ad una variabile. La funzione sarà poi invocabile tramite il nome della variabile. La chiamata può avvenire solo dopo l'assegnazione, come previsto per tutte le variabili.

```
function somma( x, y) { return x+y; }
```

```
var s = somma;
```

```
console.log( s(3,5) );
```

```
var quadrato = function(x) { return x*x };
```

```
console.log( quadrato(10) );
```

Javascript – funzioni come dati elementari

- Passare una funzione come parametro di un'altra funzione. La funzione che riceve il parametro può invocare la funzione che gli viene passata

```
function somma( x, y, logger ) {  
    var s = x+y;  
    logger(s);  
    return s;  
}  
  
function logToConsole(n) { console.log(n); }  
  
var s = somma(10,20, logToConsole);
```

Javascript – funzioni come dati elementari

- Creare funzioni anonime, ovvero senza nome, utilizzabili senza doverle salvare.

```
function somma( x, y, logger ) {  
    var s = x+y;  
    logger(s);  
    return s;  
}  
  
var logToConsole = function(n) { console.log(n); }  
  
var s = somma(10,20, logToConsole);
```

Javascript – funzioni come dati elementari

- Creare funzioni anonime, ovvero senza nome, utilizzabili senza doverle salvare.

```
var s = somma(10,20, function(n) { console.log(n); } );
```

E' possibile specificare funzioni anonime con una forma più compatta, detta lambda expression o arrow functions: La funzione anonima `function(n) { console.log(n); }` può essere riscritta come: `(n) => { console.log(n); }`

In generale una funzione anonima `function(p1,p2...,pn) { istruzioni }` può essere riscritta come:

```
(p1,p2...,pn) => { istruzioni }
```

Javascript – funzioni come dati elementari

In generale una funzione anonima `function(p1,p2...,pn) { istruzioni }` può essere riscritta come: `(p1,p2...,pn) => { istruzioni }`

Se c'è solo un parametro è possibile omettere le parentesi tonde: `p1 => { istruzioni }`

Se c'è solo una istruzione è possibile omettere le parentesi graffe: `p1 => istruzione`

Se c'è solo una istruzione il suo risultato sarà considerato come valore di ritorno della funzione anonima. Non serve specificare la parola chiave `return`

Javascript – array - Find

Alcuni metodi degli array prevedono il passaggio di una funzione. Per esempio:

- `find( )` – restituisce il primo elemento dell'array che soddisfa determinate condizioni

Il metodo `find` prevede che gli venga passato una funzione che:

- Riceve in input un singolo elemento dell'array
- Restituisce un booleano che vale `true` se l'elemento soddisfa la ricerca, altrimenti `false`

La funzione passata come parametro verrà eseguita per ogni elemento. Verrà restituito il primo elemento dell'array per il quale la funzione ha restituito `true`.

Javascript – array - Find

La funzione può essere passata in più modi:

1. Passando il nome di una funzione esistente

```
function criterioDiRicerca(element)
{
    return element > 0;
}
```

```
const array = [-1, 0, 1, 2];
const risultato = array.find(criterioDiRicerca);
```


Javascript – array - Find

La funzione può essere passata in più modi:

2. Passando una funzione anonima creata al volo

```
const array = [-1,0,1,2];  
const risultato = array.find(function(element)  
{  
    return element > 0;  
});
```

3. Passando una arrow function

```
const array = [-1,0,1,2];  
const risultato = array.find( element => element > 0 );  
const risultato = array.find( e => e > 0 );
```

Javascript – array - Sort

Alcuni metodi degli array prevedono il passaggio di una funzione. Per esempio:

- `sort( )` – ordina l'array usando, come criterio di ordinamento, la funzione passata come parametro. Se non viene passato nessun parametro l'ordinamento è alfabetico. L'ordinamento avviene direttamente sull'array e viene anche restituito un riferimento all'array stesso

Il metodo `sort` prevede che gli venga passato una funzione che:

- Riceve in input due elementi dell'array
- Restituisce un valore numerico < 0 se il primo elemento è minore del secondo, zero se sono uguali, > 0 se il primo elemento è $>$ del secondo

La funzione passata come parametro verrà eseguita per ogni coppia di elementi, fino a determinare il nuovo ordinamento dell'array

Javascript – array - Sort

La funzione può essere passata in più modi:

1. Passando il nome di una funzione esistente

```
function criterioDiOrdinamento(element1, element2)
{
    return element2 - element1;
}
```

```
const array = [-1,0,1,2];
array.sort(criterioDiOrdinamento);
```

NB: Quale ordinamento stiamo generando con questo codice?

Javascript – array - Sort

La funzione può essere passata in più modi:

2. Passando una funzione anonima creata al volo

```
const array = [-1,0,1,2];  
array.sort(function(element1,element2)  
{  
    return element2 - element1;  
});
```

3. Passando una arrow function

```
const array = [-1,0,1,2];  
array.sort( (element1,element2) => element2 - element1);
```

Javascript – array - forEach

Alcuni metodi degli array prevedono il passaggio di una funzione. Per esempio:

- `forEach( )` – esegue la funzione passata come parametro per ogni elemento dell'array, procedendo per ordine, dal primo all'ultimo elemento..

Il metodo `forEach` prevede che gli venga passato una funzione che:

- Riceve in input un elemento dell'array
- Definisce il codice da eseguire su quell'elemento

Javascript – array - forEach

La funzione può essere passata in più modi:

1. Passando il nome di una funzione esistente

```
function criterioEsecuzione(element)
{
    console.dir(element);
}
```

```
const array = [-1,0,1,2];
array.forEach(criterioEsecuzione);
```

Javascript – array - forEach

La funzione può essere passata in più modi:

2. Passando una funzione anonima creata al volo

```
const array = [-1,0,1,2];  
array.forEach(function(element)  
{  
    console.dir(element);  
});
```

3. Passando una arrow function

```
const array = [-1,0,1,2];  
array.forEach( element => console.dir(element));
```

Javascript – array - every

Alcuni metodi degli array prevedono il passaggio di una funzione. Per esempio:

- `every( )` – Determina se tutti gli elementi dell'array soddisfano la funzione passata come parametro.

Il metodo `every` prevede che gli venga passato una funzione che:

- Riceve in input un elemento dell'array
- Restituisce `true` se l'elemento soddisfa il criterio

Il metodo `every` restituisce `true` solo se la funzione ha restituito `true` per ogni elemento. Nel momento in cui un elemento restituisce `false`, quelli successivi non vengono più valutati

Javascript – array - every

La funzione può essere passata in più modi:

1. Passando il nome di una funzione esistente

```
function criterioTuttiPari(element)
{
    return element % 2 == 0
}
```

```
const array = [-1,0,1,2];
const risultato = array.every(criterioTuttiPari)
```

Javascript – array - every

La funzione può essere passata in più modi:

2. Passando una funzione anonima creata al volo

```
const array = [-1,0,1,2];  
const risultato = array.every(function(element)  
{  
    return element % 2 == 0;  
});
```

3. Passando una arrow function

```
const array = [-1,0,1,2];  
const risultato = array.every( element => element % 2 == 0);
```

Javascript – array

Alcuni metodi degli array prevedono il passaggio di una funzione. Per esempio:

- `some( )` – Determina se almeno un elemento dell'array soddisfa la funzione passata come parametro. La funzione deve avere la stessa firma prevista per `every`
- `filter( )` – Restituisce un array con tutti gli elementi che soddisfano la funzione passata come parametro. La funzione deve avere la stessa firma prevista per `find`
- `map( )` – restituisce un array in cui ogni elemento è il risultato dell'elaborazione effettuato dalla funzione passata come parametro. La funzione riceve come input un elemento dell'array (più alcuni parametri opzionali) e restituisce l'elemento opportunamente elaborato
- `reduce( )` – restituisce una "somma" calcolata su tutti gli elementi di un array. La somma viene calcolata con una funzione passata come parametro, con firma composta da due parametri, un totalizzatore e il singolo elemento dell'array. La funzione deve restituire il nuovo valore per il totale. Reduce accetta come secondo parametro opzionale il valore iniziale per il totalizzatore

Javascript – Asynchronous programming (cenni)

Javascript fetch api

- La funzione `fetch` permette di inviare una richiesta HTTP ad un server, intercettando la risposta.
- E' utilizzata per recuperare dati da API RESTFull o per inviare dati ad un backend remoto
- La chiamata è ASINCRONA, ovvero l'applicazione continua la sua esecuzione senza attendere la risposta del server
- Il risultato della chiamata sarà gestito tramite callback, promises e `async/await`

Javascript – Cenni di programmazione asincrona

- In precedenza abbiamo visto che Javascript è event based.
- Nessun codice viene eseguito se non in risposta ad un evento:
 - Deve essere "lanciato l'evento"
 - Deve esistere un "event handler"
- L'interprete javascript è inoltre single threaded:
 - Viene eseguito un event handler alla volta
 - Non è possibile eseguirne molteplici in parallelo
 - Fintanto che un event handler non è terminato non si può passare al prossimo
 - Esiste una coda degli eventi in attesa di esecuzione (event loop)

Javascript – Cenni di programmazione asincrona

- Un event handler che richiede un lungo tempo di esecuzione ritarderà l'esecuzione di tutti gli altri, di fatto bloccando l'applicazione
- In alcune circostanze si può generare il fenomeno dell'attesa attiva, ovvero un event handler che risulta in esecuzione (bloccando tutti gli altri) senza però eseguire nessuna istruzione.
- Si può verificare attesa attiva in tutte quelle situazioni in cui il codice si interfaccia con periferiche esterne. Per esempio:
 - Lettura / scrittura di un file
 - Comunicazione in rete (p.e. verso un backend remoto)

Javascript – Cenni di programmazione asincrona

- La programmazione asincrona permette di risolvere il problema dell'attesa attiva, ottenendo applicazioni più performanti e responsive.
- Nella programmazione asincrona il codice viene "diviso in due parti", quella eseguita prima e quella eseguita dopo l'attesa attiva.
- Il codice eseguito dopo l'attesa attiva viene inserito nell'event loop ed eseguito in seguito, liberando la possibilità di eseguire altri event handler

Javascript – Cenni di programmazione asincrona

- Una funzione asincrona (p.e. `fetch`) non restituisce un risultato ma una Promise, ovvero un oggetto che rappresenta un risultato che arriverà in modo asincrono in un secondo momento
- L'oggetto promise prevede tre metodi principali:
 - `then` → per specificare del codice da eseguire all'arrivo del risultato
 - `catch` → per specificare del codice da eseguire in caso di errore
 - `finally` → per specificare del codice da eseguire in ogni caso, sia di successo che di errore
- Ognuno dei metodi precedenti restituisce a sua volta una Promise, permettendo di concatenarli tra di loro

Javascript fetch api – Esempi di richiesta in GET

```
fetch('http://example.com')  
  .then(response => console.log(response))  
  .catch(error => console.log('Errore'));
```

L'oggetto Response contiene alcuni metodi utili per recuperare i dati ricevuti con la risposta. `text()` e `json()` restituiscono i dati rispettivamente in forma testuale e in forma di oggetto json. Entrambi sono asincroni e restituiscono una promise

```
fetch('http://example.com')  
  .then(response => response.json())  
  .then(response => console.log(response))  
  .catch(error => console.log('Errore'));
```

Javascript fetch api – Esempio di richiesta in POST

```
fetch('http://example.com', {  
    method: 'post',  
    body: JSON.stringify({ username: 'admin', pwd: '123456' })  
})  
  
.then(response => response.json())  
  
.then(data => console.log('Successo:', data))  
  
.catch(error => console.log('Errore'));
```

Javascript – Cenni di programmazione asincrona – async / await

- La programmazione asincrona può essere resa più leggibile utilizzando le parole chiave `async` e `await`, tramite le quali il codice asincrono appare simile al codice sincrono
- Il codice asincrono può essere scritto con la stessa sintassi del codice sincrono, a patto di:
 - Utilizzare la parola chiave `await` prima della chiamata della funzione asincrona
 - Inserire la parola chiave `async` prima della funzione dentro la quale si utilizza la parola chiave `await`
 - Gestire gli errori con i classici `try / catch`

Javascript – Cenni di programmazione asincrona – async / await

```
async function AsyncFunction()  
{  
    let response = await fetch('http://example.com')  
    let json = await response.json()  
    console.log(json)  
}
```